

MusicOS：プログラマブルな音楽権利と流通のためのパーミッションレス・プロトコル

Version 1.0 — 2026 年 2 月

Subsonic Labs
os@subsoniclabs.io

概要 音楽産業は、物理的な流通時代に構築されたインフラストラクチャの上で運営されている。不透明なロイヤリティ・パイプライン、断片化された所有権データベース、そして創造的貢献よりもポジショナル・コントロール（構造的支配力）からその価値を引き出す仲介者たち。メタデータの不備や矛盾により、年間推定 25 億ドルものロイヤリティが未請求のまま放置されている。一方で、アーティストは消費から支払いに至る収益の流れを構造的に検証することができない。生成 AI の台頭はこの問題をさらに深刻化させ、音声制作そのものをコモディティ化し、アーティストが流通から獲得する本来少ない利益率をさらに侵食している。

本稿では MusicOS を提示する。MusicOS は、Sui ブロックチェーン上のパーミッションレス・プロトコルであり、楽曲、録音、リリース、そしてそれらの創作に関わる当事者のための包括的なオンチェーン・データモデルを提供する。MusicOS は「プログラマブル・ミュージック」を導入し、「オープンソース・ミュージック」の理念を体現する。各楽曲、録音、リリースは公開台帳上の正規のアドレス可能なオブジェクトであり、時間とともに進化し、メディアやメタデータを蓄積し、その完全な状態をあらゆる利用者に公開する。録音には最終マスターだけでなく個々のステム（録音のソースコード）が含まれ、音楽の創造的な構成要素をオープンかつプログラマブルに提供する。音楽権利の完全な構造（楽曲と録音の区別、多者間のクレジット帰属、ベースポイント収益分配、シェアトークン所有権）をスマートコントラクトに直接エンコードすることにより、MusicOS はオープンネスの経済がコントロールの経済を上回る、検証可能、決定論的、かつエージェント互換の流通レイヤーを創出する。

目次

1 はじめに	3
2 背景と動機	3
2.1 音楽権利の構造	3
2.2 現行産業のインフラストラクチャ障害	4
2.3 音声のコモディティ化	5
2.4 オープンソース・ミュージック	5
2.5 既存のブロックチェーンアプローチの限界	6
3 プロトコル設計	7
3.1 設計原則	7
3.2 コアオブジェクト	7
3.3 オブジェクトのライフサイクルと状態マシン	8
3.4 パーティーとクレジットシステム	9
3.5 音声インジェスト	9
3.6 決定論的オブジェクトアドレッシング	10
4 エクステンション・システム	10
4.1 動機：WordPress モデル	10
4.2 登録とウィットネスゲーテッド・アクセス	10
4.3 生 UID アクセスと将来互換性	11
4.4 エクステンションの分離と信頼モデル	11
5 収益分配とトークンエコノミクス	11
5.1 シェアトークン	11
5.2 ベーシスポイント収益分配	12
5.3 リリース収益分配	12
5.4 報酬プール	12
5.5 マルチカレンシー対応	13
6 デイール・プロトコル	13
7 分散型メディアストレージ	13
8 エージェント互換性とプログラマブル・ミュージック	14
8.1 DDEX メッセージングから宣言的実行へ	14
8.2 エージェントネイティブ・インターフェース	14
8.3 新しい流通フォーマットとしてのプログラマブル・ミュージック	15
9 関連研究との比較	15
10 議論	16
10.1 信頼の前提	16
10.2 プロトコルの中立性	17
10.3 なぜ Sui なのか	17
10.4 オンチェーンコスト分析	18
10.5 制限とトレードオフ	18
11 結論	19

1 はじめに

世界の録音音楽産業は 2023 年に 286 億ドルの収益を生み出した¹。しかし、クリエイターをその収益に結びつけるインフラストラクチャは驚くほど原始的なままである。所有権データは、著作権管理団体 (PRO)、機械的ライセンス機関、出版社、レーベル、ディストリビューターが管理する断絶されたデータベースに散在しており、各組織は不完全でしばしば矛盾するレコードを保持している。楽曲がストリーミングされると、それが生み出す収益は一連の仲介者を通過し、各仲介者が権利保持者自身が独立して検証できない独自の計算を適用する。その結果、音楽を創造する人々が、それがどのように収益化されているかについて構造的に最も情報を持たない産業が出来上がっている。

本稿では、Sui ブロックチェーン上に構築されたパーミッションレス・プロトコルである MusicOS を紹介する。MusicOS は音楽権利管理と流通のための基盤データモデルおよび経済ロジックを提供する。MusicOS は音楽産業の参加者 (アーティスト、レーベル、出版社、ディストリビューター、プラットフォーム) を置き換えることを目的としていない。それらの参加者が運営する「インフラストラクチャ」を置き換えることを目的としている。この区別は重要である。MusicOS はアプリケーションではなくプロトコルであり、プラットフォームではなく基盤である。

本プロトコルは 3 つの主要な貢献を行う。

1. **音楽権利のための忠実なオンチェーン・データモデル。** MusicOS は、音楽産業の権利の実際の構造 (楽曲 (著作物) と録音 (音声パフォーマンス) の法的区別、役割別分類による多者間クレジット帰属、階層的収益分配) をエンコードする。トークンゲートッド・アクセスや NFT コレクティブルに単純化するのではなく。
2. **パーミッションレスなエクステンション・アーキテクチャ。** WordPress に着想を得て、MusicOS はいかなるデベロッパーも、コアプロ

トコルの変更や許可を必要とせずに、コアプロトコルオブジェクトに新しい機能を追加するエクステンションパッケージをデプロイできるようにする。エクステンションはウィットネスゲートッドUID アクセスを通じて親オブジェクトと相互作用し、コアプロトコルのアップグレードなしに新しい Sui フレームワークプリミティブの即座の採用を可能にする。

3. **流通フォーマットとしてのプログラマブル・ミュージック。** 分散型ストレージ (Walrus) に保存された音声ファイルをオンチェーンの経済ロジック (所有権シェア、収益分配、クレジット帰属、エクステンション定義の振る舞い) にバインドすることにより、MusicOS は新しい種類の流通可能なアーティファクトを創出する。音楽とその経済が不可分であり、機械可読である。

本稿の残りは以下のように構成される。節 2 では音楽権利の構造と MusicOS の動機となるインフラストラクチャの障害について背景を説明する。節 3 ではコアプロトコルの設計を提示する。節 4 ではエクステンション・システムの詳細を述べる。節 5 では収益分配モデルとトークンエコノミクスについて説明する。節 6 では録音からリリースへの承認のためのディール・プロトコルを取り上げる。節 7 では Walrus を介した分散型メディアストレージについて論じる。節 8 ではエージェント互換性とプログラマブル・ミュージックの概念を紹介する。節 9 では MusicOS を関連研究と比較する。節 10 では信頼の前提と制限について議論する。

2 背景と動機

2.1 音楽権利の構造

音楽産業のインフラストラクチャがなぜ壊れているかを理解するには、まず音楽権利が実際にどのように機能するかを理解しなければならない。録音されたすべての音楽には、少なくとも 2 つの異なる権利セットが関与している。

¹IFPI. Global Music Report 2024. International Federation of the Phonographic Industry, 2024.

楽曲の権利 (Composition rights) は、基礎となる著作物（メロディー、ハーモニー、歌詞）を対象とする。これらはソングライターと出版社が保持する。1つの楽曲は複数の録音（カバーバージョン、リミックス、ライブパフォーマンス）を持つことができ、各録音は楽曲の権利保持者と録音の権利保持者の間で分配されなければならない収益を生み出す。

録音の権利 (Recording rights)（しばしば「マスター権」と呼ばれる）は、楽曲の特定の音声パフォーマンスを対象とする。これらは演奏アーティストとレーベルが保持する。録音は基礎となる楽曲なしには存在できず、それが生み出す収益は楽曲側と録音側の間で事前に交渉された分配率に従う。

リリース (Release)（アルバム、EP、シングル）は、複数の録音を流通可能な製品に集約する。リリース上の各録音は、異なる権利保持者、異なる楽曲・録音分配率、異なる収益配分を持つ場合がある。

この三層構造（楽曲、録音、リリース）は実装の詳細ではない。これは音楽産業の法的・経済的現実であり、事実上すべての法域で著作権法に成文化されている。音楽権利を表現すると称するいかなるプロトコルも、この構造を忠実にモデル化しなければならない。これを「楽曲1つにつき NFT1つ」に平坦化することは、産業の経済的関係が依存する情報を破棄することになる。

2.2 現行産業のインフラストラクチャ障害

音楽産業のインフラストラクチャ障害は相互に関連し、自己強化的である。

ブラックボックス問題。 リスナーが Spotify、Apple Music、その他のデジタルサービスプロバイダー（DSP）で楽曲をストリーミングすると、そのストリームが生み出す収益は、権利保持者が独立して監査できないパイプラインに入る。DSP は利用状況をディストリビューターに報告し、ディストリビューターはレーベルに報告し、レーベルはアーティストに報告し、各ステップでレイテンシ、不透明性、そしてエラーまたは搾取の可能性が導入される。四半期ごとのロイヤリティ明細書を受け取る

アーティストには、基礎となる消費データに対してその正確性を検証するメカニズムがない。

未マッチング・ロイヤリティ危機。 2018 年の音楽近代化法（Music Modernization Act）は、米国において Mechanical Licensing Collective (MLC) を設立し、未マッチングの機械的ロイヤリティ（所有権データが不完全または矛盾しているために配布できないソングライターや出版社への支払い）の問題に対処した。最近の報告によれば、MLC は数億ドルもの未マッチング・ロイヤリティを保有している。² この危機は、単一の、権威ある、公開検証可能な所有権データソースを一度も持ったことのない産業の予測可能な帰結である。

障壁としてのライセンシング複雑性。 楽曲の権利と録音の権利の区別は、音楽を商業利用しようとするいかなる主体も、複数の組織にまたがる複数の権利保持者と交渉しなければならないことを意味する。PRO（ASCAP、BMI、SESAC）は楽曲の公演権を扱う。MLC は機械的権利を扱う。レーベルはマスター使用ライセンスを扱う。出版社はシンクロナイゼーション権を扱う。各組織は独自のデータベース、独自の支払いスケジュール、独自の料金体系で運営している。その結果、単一のユースケースで単一の楽曲をライセンスするだけでも、共有インフラストラクチャのない半ダースの組織をナビゲートする必要があるシステムが出来上がっている。

データの断片化。 音楽所有権の正規のグローバルデータベースは存在しない。国際標準レコーディングコード（ISRC）は録音を識別し、国際標準楽曲コード（ISWC）は楽曲を識別するが、これらの識別子は一貫性なく適用され、頻繁に重複し、所有権や支払い情報にプログラム的にリンクされていない。ストリームを支払いに結びつけるメタデータは、相互運用できないプロプライエタリなデータベースで管理されている。

MusicOS は、コアオブジェクトにレガシー識別子を混入させることなくブリッジング問題に対処する。Sui のオンチェーン・ネームサービスである SuiNS³は、自然な解決レイヤーを提供す

²U.S. Copyright Office. Mechanical Licensing Collective: Annual Report. Library of Congress, 2023.

³SuiNS. Sui Name Service. <https://suins.io>.

る。例えば、ISWC コード `T-123.456.789-Z` は `t123456789z.iswc.sui` として表現し、対応する Composition オブジェクトにポイントできる。同様に、ISRC は Recording に解決でき、その他のあらゆる業界コード (IPI, ISNI など) も独自の SuiNS サブドメインを通じてマッピングできる。オペレーターはレガシーコードをオンチェーンの対応物に解決する検証済みネームサービスを運営でき、コアプロトコルオブジェクトの変更やオフチェーンインデクシングまたはカスタムオンチェーンレジストリへの依存なしに、既存の業界インフラストラクチャとの相互運用性を提供する。

2.3 音声のコモディティ化

これらのインフラストラクチャ障害は、加速する変化を背景に存在している。生成 AI は音声制作の限界費用をゼロに向かって低下させた。AI 支援の作曲、編曲、マスタリングのツールは広く利用可能であり、急速に改善されている。アーティストへの影響は明白である。もし流通の主要単位が音声ファイルであり、音声ファイルがほぼ無コストで制作可能になりつつあるならば、流通パイプライン内における人間のクリエイターの経済的地位は侵食され続けるであろう。

対応策は、音楽の流通が何を意味するかを拡張することである。音声だけがコモディティである場合、人間のアーティストが貢献する価値 (創造的意図、文化的文脈、協調的關係、作品を取り巻く社会的・経済的ネットワーク) は流通フォーマット自体に捕捉されなければならない。サーバー上にある音声ファイルはこれらのいずれも捕捉しない。音声とその所有構造、クレジット帰属、収益ロジック、拡張可能なメタデータにバインドするプログラマブルなオンチェーンオブジェクトは、これらのすべてを捕捉する。

MusicOS は、音楽流通の単位は音声ファイルであるべきではないと提唱する。それは「プログラマブルな楽曲-録音-リリース構造」、すなわち楽曲とそれが体現するすべての関係の機械可読で経済的に完全な表現であるべきである。

2.4 オープンソース・ミュージック

オープンソースソフトウェアは世界を動かしている。グローバルなデジタル経済を支えるオペレーティングシステム、ウェブサーバー、データベース、プログラミング言語、AI フレームワークは、圧倒的にオープンソースである。そのロジックはよく理解されている。ソースコードが透明で、検査可能で、コンポーザブルであるとき、クローズドなシステムでは実現できない派生的イノベーションの組み合わせ的爆発を可能にする。

MusicOS はこのロジックを音楽に適用する。プロトコルは、録音に最終マスター音声だけでなく、個別のステム (録音の「ソースコード」を構成する分離されたボーカル、楽器、プロダクションレイヤー) を含めることを要求する。従来のストリーミングプラットフォームは「コンパイル済みバイナリ」のみを配信する。つまり、消費者が聴くことはできるが、有意義に検査、分解、またはその上に構築することのできないミックスダウンされた音声ファイルである。MusicOS はソースを配信する。

音楽産業に精通している者にとって、これは急進的な提案である。ステムは「絶対に共有してはならないもの」である。マネージャー、レーベル、そして苦い経験によってアーティストに叩き込まれた常識は、ステム、セッション、生の素材を守れということである。なぜなら、従来の産業におけるあらゆる関係は根本的に敵対的だからである。レーベルはアーティストが稼ぐ前にリクープする。出版社はソングライターが明細書を見る前に回収する。ディストリビューターはすべての受け渡しで手数料を抽出する。この環境では、作品の構成要素を共有することは不合理である。経済的利益が構造的に不一致な当事者にレバレッジを与えてしまうからだ。

MusicOS はこの計算を変える。不透明な仲介者を決定論的なオンチェーンロジック (収益分配率は密室で交渉されるのではなくエンコードされ、所有権は契約を保持する者が主張するのではなく検証される) に置き換えることにより、プロトコルはステムの秘匿を合理的にしている敵対的条件を除去する。アーティストがすべてのドルの流れを正確に検証でき、シェアトークンが監査不能なロイヤ

リティ明細書ではなく直接的な経済参加を与え、エクステンションが提供したステムそのものから新たな収益源を解放できるとき、オープンネスのROIはコントロールのROIを上回る。

これがMusicOSの核心にある意図的なトレードである。音楽をオープンソース化することと引き換えに、アーティストは自分の作品が完全にプログラマブルであるオンチェーン経済へのアクセスを得る。ステムがオンチェーンでアドレス可能であるとき、音楽作品の上に構築できるプロダクトの範囲は劇的に拡大する。リミックスツール、AI支援のプロダクションワークフロー、インタラクティブなリスニング体験、ステムレベルのライセンスング、教育アプリケーション、そしてまったく新しいクリエイティブフォーマットが可能になる。プラットフォームとのアドホックなパートナーシップを通じてではなく、誰でも構築できるパーミッションレスなエクステンションとして。ステムは録音のAPIサーフェスである。

オープンソースソフトウェアは、透明性とコンポーザビリティが捕捉する以上の経済的価値を創出することを実証した。オープンソース・ミュージックは同じ提案である。音楽の創造的構成要素をオープンに利用可能かつプログラマブルにすることにより、MusicOSはクローズドな流通では不可能な方法で、クリエイター、デベロッパー、リスナーに利益をもたらす派生的価値のエコシステムを可能にする。その名前自体がこの二重のアイデンティティをエンコードしている。MusicOSは音楽のためのオペレーティングシステム（アプリケーションが構築される基盤レイヤー）であると同時に、流通モデルとしてのオープンソースへのコミットメントである。

2.5 既存のブロックチェーンアプローチの限界

近年、いくつかのブロックチェーンベースの音楽プロジェクトが登場し、上述の問題のいくつかの側面に対処している。Audiusは分散型ストリーミングプラットフォームを提供する。Sound.xyzはアーティストが限定版の録音をNFTとして販売できる

ようにする。Royalはファンがストリーミングロイヤリティの端数的権利を購入できるようにする。これらのプロジェクトはブロックチェーンベースの音楽インフラストラクチャへの需要を実証したが、共通の限界を共有している。

プラットフォームであってプロトコルではない。ほとんどのブロックチェーン音楽プロジェクトは、独自のユーザーインターフェース、トークンエコノミクス、ガバナンス構造を持つアプリケーションである。それらは目的地であり、インフラストラクチャではない。Audiusを使用するアーティストはAudiusを使用しているのであり、いかなるアプリケーションも統合できるパーミッションレス・プロトコルの上に構築しているのではない。

簡素化されたデータモデル。既存のプロジェクトは通常、音楽を楽曲・録音・リリースの階層を忠実に表現するのではなく、単一のアセット（NFT、トークン、ストリーム）としてモデル化する。この簡素化はコレクティブルやファンエンゲージメントには適切かもしれないが、音楽産業の全範囲の運営（機械的ライセンスング、シンクロナイズーション権、多者間収益分配、組織間データ交換）をサポートすることはできない。

許可制のエクステンション。既存のブロックチェーン音楽プラットフォームに新しい機能を追加するには、通常プラットフォームの協力が必要である。独立した開発者がコアコードベースを変更したりメンテナの承認を得たりすることなく、プロトコルの機能を拡張するメカニズムはない。

MusicOSはこれらの限界のそれぞれに対処する。MusicOSはプラットフォームではなくプロトコルである。音楽権利の完全な構造をモデル化する。そして、そのエクステンション・システムは誰もが許可なくその上に構築できるようにする。

3 プロトコル設計

3.1 設計原則

MusicOS は、Move プログラミング言語⁴を用いて Sui ブロックチェーン上に構築されている。プロトコルの設計は 4 つの原則に基づいている。

現実世界の権利構造への忠実性。 オンチェーン・データモデルは、音楽権利を統治する法的・経済的関係を忠実に表現しなければならない。現実世界の運営に必要な情報を破棄する簡素化は、完全性を優先して拒否される。

公開後の不変性。 コア音楽メタデータ（楽曲、録音、リリース）は、可変（Initialized）から不変（Published）への状態マシンに従う。一度公開されると、オブジェクトは Sui 上の共有オブジェクトとなり、誰でも読み取り可能で誰も変更不可能となる。これにより、音楽作品の権威あるレコードが遡及的に改変されないことが保証される。

ケーパビリティベースの認可。 MusicOS オブジェクトに対するすべての可変操作には、対応する管理ケーパビリティ（`CompositionAdminCap`、`RecordingAdminCap`、`ReleaseAdminCap`、`PartyAdminCap`）が必要である。ケーパビリティは標準的な Sui オブジェクトであり、転送、共有、または保管できるため、認可ロジックをプロトコルにハードコーディングすることなく柔軟なガバナンスモデルを可能にする。

許可なき拡張性。 コアプロトコルはデータモデルとライフサイクル管理を提供する。すべての追加機能（収益分配、報酬プール、メタデータ拡充、ライセンス自動化）は、いかなるデベロッパーも独立してデプロイできるエクステンションとして実装される。

3.2 コアオブジェクト

MusicOS は 4 つの主要なオンチェーンオブジェクトを定義する。

Party（パーティー）。 音楽の創作または管理に関与するあらゆるエンティティを表現する。個人のアーティスト、プロデューサー、エンジニア、またはグループ（バンド、オーケストラ、合唱団）が含まれる。グループは個々のパーティーメンバーへの参照を含むことができる。パーティーは MusicOS における基盤的なアイデンティティオブジェクトであり、楽曲、録音、リリースはすべてクレジットマップにおいてパーティーを参照する。

```
Party {
  id: UID,
  kind: Individual | Group(VecSet<ID>),
  name: String,           // 最大200バイト、最大200メンバー
}
```

Composition<S>（楽曲）。 基礎となる著作物（メロディー、歌詞、ハーモニー）を表現する。楽曲をその所有権トークンにリンクするファントム・シェアトークン型 `S` でパラメータ化される。タイトル、別タイトル、クレジット（パーティー ID から役割へのマッピング）、ベースポイント収益分配率、および Walrus に保存されたオプションのコンテンツ参照（歌詞、コード譜、スコア、デモ音声）を含む。

```
Composition<S> {
  id: UID,
  state: Initialized | Published(timestamp),
  title: String,
  alternate_titles:
    vector<String>,      // 最大5
  credits: VecMap<ID>,
  Credit<Role>>,        // 最大50
  split_bps: BPS,
  lyrics: Option<WalrusData>,
  chart: Option<WalrusData>,
  score: Option<WalrusData>,
  demo: Option<Audio>,
}
```

Recording<S>（録音）。 楽曲の音声パフォーマンスを表現する。豊富な音楽メタデータ（ジャンル、

⁴Sui Foundation. The Move Programming Language. 2024.

キー、テンポ、拍子)、マスター音声ファイル、個別の音声ステム(最低2つ)、カバーアート、23の役割タイプによるクレジット帰属を含む。各録音は親楽曲を参照し、作成時に楽曲の収益分配率を捕捉する。

```
Recording<S> {
  id: UID,
  state: Initialized |
Published(timestamp),
  title: String,
  composition_id: ID,
  composition_split_bps: BPS,
  primary_genre_id: ID,
  secondary_genre_ids: VecSet<ID>, //
  最大3
  primary_artist_ids: VecSet<ID>, //
  最大20
  featured_artist_ids: VecSet<ID>, //
  最大50
  credits: VecMap<ID, Credit<Role>>, //
  最大150
  language: Option<LanguageCode>,
  is_explicit: bool,
  is_instrumental: bool,
  lyrics: Option<WalrusData>,
  musical_key: Option<MusicalKey>,
  time_signature: Option<TimeSignature>,
  tempo_bpm: Option<u16>,
  master: Audio, //
  必須
  stems: vector<Stem>, //
  最低2、最大100
  cover_art: CoverArt,
}
```

Release(リリース)。録音の流通可能なコレクション(アルバム、EP、シングル)を表現する。トラックをディスクに整理し、リリースレベルのクレジット(Primary、Featured アーティスト)をサポートし、入金をトラックおよびその基礎となる楽曲・録音に分配する収益分配ロジックを含む。

```
Release {
  id: UID,
  kind: Album | EP | Single,
  state: Initialized(bool) |
Published(u64),
```

```
  title: String,
  subtitle: Option<String>,
  description: String,
  credits: VecMap<ID, Credit<Role>>, //
  最大50
  discs: vector<Disc>, //
  最大20
  cover_art: CoverArt,
}
```

3.3 オブジェクトのライフサイクルと状態マシン

楽曲、録音、リリースは二状態のライフサイクルに従う。

Initialized → Published

Initialized 状態では、オブジェクトは作成者に所有され、自由に変更できる。クレジットの追加、メタデータの設定、コンテンツの添付などが可能である。すべての変更関数はステートガードを適用し、公開済みオブジェクトに対して呼び出された場合はアボートする。

`publish()` の遷移は不可逆である。完全性制約(楽曲は少なくとも1つのクレジットと1つのコンテンツ添付を持つ必要がある。録音はクレジット、プライマリーアーティスト、少なくとも2つのステムを持つ必要がある。リリースはプライマリークレジットと有効なトラック割り当てを持つ必要がある)を検証し、Sui Clock から公開タイムスタンプを記録し、オブジェクトを Sui 共有オブジェクトに変換し、公開的に読み取り可能で永続的に不変とする。

このライフサイクルにより、公開された音楽メタデータは永続的で改ざん不可能なレコードを構成する。エクステンションはダイナミックフィールドを通じて公開済みオブジェクトに追加データを添付できるが、コアメタデータは固定される。

3.4 パーティーとクレジットシステム

音楽産業における帰属は、単純な名前のリストではない。録音には通常、数十のクレジットされた当事者が関与し、それぞれが特定の役割を持つ。プロデューサー、ボーカリスト、ミキシングエンジニア、マスタリングエンジニア、楽器奏者など、多数の役割がある。MusicOS はこれをジェネリックなクレジットシステムでモデル化する。

```
Credit<Role> {
  roles: vector<Role>,
}
```

Role 型パラメータはコンテキストごとに特殊化される。

- **CompositionPartyRole:** Composer、Lyricist、Songwriter、Arranger、Adapter、Translator
- **RecordingPartyRole:** Producer、Vocalist、MixingEngineer、MasteringEngineer、Instrumentalist、Conductor など 23 の役割バリエーション。多くの役割はオプションのレベル修飾子 (Lead、Featured、Assistant 等) をサポートする。
- **ReleasePartyRole:** Primary、Featured

パーティーは、プライマリーまたはフィーチャードアーティストとして指定される前に、録音にクレジットされていなければならない。ステム提供者は、その貢献が登録される前に録音にクレジットされていなければならない。これらの参照整合性制約により、クレジットグラフが内部的に一貫していることが保証される。

3.5 音声インジェスト

MusicOS における音声は、「ウィットネスゲテッド・インジェスト」を通じて作成される。`audio::new()` 関数は、`drop` アビリティを持つジェネリック・ウィットネス型パラメータ `Ingestor` を必要とする。型を構築できるのはそれを定義した Move モジュールのみであるため、すべての `Audio` 構造体にはどのインジェスター

パッケージが作成したかを特定するオンチェーン証明が付与される。MusicOS はパーミッションレスであり (誰でもインジェスターを構築しデプロイできる)、ウィットネス型は各 `Audio` にその出所を永続的に刻印し、下流の利用者が情報に基づいた信頼の判断を行えるようにする。

```
public fun new<Ingestor: drop>(
  channels, bit_depth, sample_rate_hz,
  samples, data, _ingester: Ingestor,
) -> Audio
```

インジェスターのウィットネス型は結果の `Audio` 構造体に `TypeName` として記録され、どのインジェスターが音声データとその技術パラメータ (チャンネル、ビット深度、サンプルレート、サンプル数) を証明したかの永続的なオンチェーンレコードを作成する。

インジェストの目的は、指定された Walrus Blob ID でアドレスされた音声ファイルが、申告された音声メタデータに対応していることを証明することである。すなわち、その Blob ID のファイルが実際に申告されたチャンネル数、ビット深度、サンプルレート、サンプル数を持つことを証明する。この証明がなければ、メタデータが保存された音声データを正確に記述しているというオンチェーンの保証は存在しない。

このパターンはプラグラブルなインジェストを可能にする。異なるインジェスターモジュールが異なる検証戦略を実装でき、コアプロトコルは音声を作成される前に「何らかの」インジェスターが証明を行ったことを保証する。例えば、Nautilus⁵ (信頼された実行環境における検証済み計算のためのフレームワーク) を利用したインジェスターは、Walrus から音声ファイルをダウンロードし、ファイルデータから Blob ID を再計算し、ファイルのストリームメタデータから音声プロパティを直接抽出し、Blob ID と検証済みパラメータを結びつける暗号的証明に署名できる。インジェスターのウィットネス型は結果の `Audio` 構造体に対する承認印として機能し、MusicOS の上位利用者 (ア

⁵Nautilus. Trustless Off-Chain Computation for Sui. <https://www.sui.io/nautilus>.

アプリケーション、プラットフォーム、エージェント)は、どのインジェスターが特定の音声ファイルを証明したかに基づいて信頼の判断を下すことができる。

3.6 決定論的オブジェクトアドレッシング

MusicOS は、Sui の派生オブジェクトメカニズムを使用して、管理ケーパビリティと録音の決定論的地址を生成する。CompositionAdminCap は親楽曲の UID から派生するため、ケーパビリティのアドレスは楽曲のアドレスのみからクライアント側で計算でき、インデクサークエリは不要である。

同様に、録音のアドレスは、マスター音声のインジェスター型を派生キーとして使用して、親楽曲の UID から派生する。これにより、楽曲からその録音への発見可能で決定論的なマッピングが作成される。

リリースはコンテンツアドレッシング派生を使用する。リリース UID は、録音 ID、トラック分配値、作成エポックの Blake2b-256 ハッシュから派生する。これにより、同じ録音セットが同じ分配率で決定論的なリリースアドレスを生成する。

4 エクステンション・システム

4.1 動機：WordPress モデル

MusicOS は WordPress のように設計されている。オープンソースのセルフホスト型プロトコルであり、必要なプラグインを自由にインストールできる。管理ケーパビリティの保持者がサイト管理者であり、登録するエクステンションを選択し、その選択の信頼性の影響を受け入れる。

この設計は意図的な哲学を反映している。コアプロトコルはデータモデルとライフサイクル管理を提供し、すべてのアプリケーション固有の振る舞いはエクステンションに存在する。収益分配、報酬プール、ライセンス自動化、メタデータ拡充、ディスカバリーアルゴリズム、ソーシャル機能はすべてエクステンションの関心事である。コアプロト

コルはフックを提供し、エコシステムが機能を提供する。

管理型 WordPress ホスティングプロバイダーに例えられる、より管理されたレイヤーは MusicOS の上に構築できる。このレイヤーは管理ケーパビリティの保管とエクステンションのホワイトリスト適用を処理し、ガードレールを望む参加者にガードレールを提供しつつ、プロトコルのパーミッションレスな性質を制限しない。

4.2 登録とウィットネスゲートッド・アクセス

エクステンションはオブジェクトごとに登録される。オブジェクトの所有者（管理ケーパビリティの保持者）は、3つの引数で register_extension を呼び出す。管理ケーパビリティ（所有権の証明）、エクステンション・ウィットネス（エクステンションモジュールの識別）、設定値（エクステンション固有の設定）。

```
composition.register_extension(  
    &composition_admin_cap,  
    my_extension::Extension(),  
    config,  
);
```

登録後、エクステンションは uid_mut_with_extension を通じて親オブジェクトの UID にミュータブルにアクセスできる。これは2つのチェックを実行する。呼び出し元がエクステンションのウィットネス型を提供していること（定義モジュールのみが構築可能）、およびエクステンションがこの特定のオブジェクトに登録されていること。

```
public fun uid_mut_with_extension<E:  
drop>(  
    self: &mut Composition<S>,  
    _extension: E,  
): &mut UID {  
    extension::assert_registered<E>(&self.id);  
    &mut self.id  
}
```

4.3 生 UID アクセスと将来互換性

MusicOS における重要な設計上の決定は、エクステンションが制限された API サーフェスではなく、生の `&mut UID` アクセスを親オブジェクトに対して受け取ることである。これは意図的である。Sui のコアプリミティブ（ファンドアキュムレーター、派生オブジェクト、コイン受信、ダイナミックフィールド）はすべて `&mut UID` で動作する。新しいプリミティブは Sui フレームワークに定期的に追加される。MusicOS が各 UID 操作をコアモジュール上の委譲関数にラップした場合、すべての新しい Sui プリミティブはエクステンションが使用する前にコアプロトコルのアップグレードを必要とする。

生 UID アクセスにより、エクステンションはコアパッケージを待つことなく、依存することなく、新しい Sui 機能を即座に採用できる。プロトコルはブロックチェーンと共に進化し、遅れを取らない。

4.4 エクステンションの分離と信頼モデル

生 UID アクセスのトレードオフは、登録されたエクステンションが親の UID に広範なアクセスを持ち、理論的には他のエクステンションのダイナミックフィールドに干渉する可能性があることである。これはパーミッションレスモデルの受け入れられた帰結である。

エクステンションの分離は、強制ではなく慣例によって達成される。

- 各エクステンションの設定は、ファントムパラメータ化されたダイナミックフィールドキー `Extension<phantom E: drop>()` を使用して保存される。ファントム型 `E` により、各エクステンションが一意のストレージスロットを持つことが保証される。
- エクステンションはオブジェクトのライフサイクル状態に関係なく、登録および登録解除できる。これは意図的であり、報酬プールのようなエクステンションは公開済み（共有）オブジェクト上で動作するように設計されている。

- `register` および `unregister` 関数は `public(package)` であり、各コアオブジェクトのゲートドエントリポイントを通じて MusicOS コアパッケージのみがエクステンションキーを追加または削除できることを保証する。

信頼モデルは明示的である。管理ケーパビリティの保持者は、登録するエクステンションを評価する責任がある。これは、WordPress の管理者がインストールするプラグインを選択する現実世界の責任を反映している。

5 収益分配とトークンエコノミクス

5.1 シェアトークン

各楽曲と録音は、作成時に独自のファンジブル・シェアトークンをミントする。これらのトークンは、収益の分配方法を決定する所有権ステークを表現する。

パラメータ：

- 供給量：10,000,000.000000 トークン（固定、初期化後に不変）
- 小数点：6（100 万分の 1 の精度まで端数の所有権を可能にする）
- シンボル：SHARE

シェアトークンは、通貨設定の正確性を保証するバリデーションパイプラインを通じて初期化される。

`Currency` オブジェクトは正確に 6 桁の小数点を持ち、シンボルは“SHARE”でなければならない、メタデータケーパビリティは削除されていなければならない（後からの変更を防止）、ミント前の供給量はゼロでなければならない。全供給量のミント後、トレジャリーケーパビリティは消費され、供給量は永久に固定される。

シェアトークンは標準的な Sui ファンジブルトークンである。転送、分割、統合、保管が可能であり、またベスティングスケジュール、ロックアップ、自動マーケットメーカーなど、エクステンシ

ン定義のロジックを含むあらゆるオンチェーンロジックによって管理できる。

5.2 ベーシスポイント収益分配

MusicOS におけるすべての収益率はベーシスポイント (BPS) で表現される。1 BPS = 0.01%、10,000 BPS = 100%。この慣例は金融サービスにおいて標準的であり、浮動小数点演算を導入することなく、音楽産業の収益分配に十分な精度を提供する。

2 種類の分配が収益フローを統治する。

楽曲分配 (Composition split) (`split_bps`)。
楽曲に設定され、トラックの収益の何パーセントが楽曲側と録音側に分配されるかを決定する。例えば、5,000 BPS の分配はトラック収益の 50% が楽曲シェア保持者に、50% が録音シェア保持者に流れることを意味する。この分配率は録音の作成時に捕捉され、楽曲所有者による遡及的変更を防止する。

トラック分配 (Track split) (`split_bps`)。
トラックがリリースに追加される際に設定され、リリースの総収益の何パーセントが各トラックに分配されるかを決定する。リリース上のすべてのトラック分配は正確に 10,000 BPS (100%) に合計されなければならない、リリース作成時に強制される。

5.3 リリース収益分配

リリースで収益が分配される場合、プロトコルは決定論的な二段階分配を実行する。



図 1: 二段階収益分配フロー

このフローの重要な基盤は、楽曲、録音、リリースの「オブジェクト性」である。各エンティティは独自のアドレスを持つファーストクラスの Sui オブジェクトであるため、リリースは `send_funds()` を介して楽曲や録音のファンドアキュムレーターに直接資金を転送でき、中間ルーティング、カスタディアルアカウント、オフチェーン決済は不要である。楽曲および録音オブジェクトは、いかなる Sui アドレスがアセットを受け取るのと同じ方法で、直接的かつ検証可能に収益を受け取る。

資金が楽曲または録音に到着すると、報酬プールエクステンションを通じたシェアトークンベースの請求に利用可能になる。リリースレベルの収益からシェア保持者ごとの請求まで、フロー全体がオンチェーンで、決定論的で、独立して検証可能である。

収益分配イベント、報酬プール預入、シェアトークン請求はすべて Sui の `emit_authenticated` プリミティブ⁶を使用して発行される。標準的なイベントとは異なり、認証済みイベントは定義元の Move パッケージのみが発行でき、発生源の暗号学的証明を提供する。ライトクライアントやオフチェーンインデクサーが `ReleaseRevenueDistributedEvent` を観察した場合、それが MusicOS のリリースモジュールによって確実に発行されたものであり、なりすましコントラクトによるものではないことを検証できる。この特性は、プロトコル上に信頼性のある金融レポーティングおよび監査インフラストラクチャを構築するために不可欠である。

5.4 報酬プール

楽曲および録音の報酬プールエクステンションは、シェアトークン保持者への比例的な収益分配を可能にする。

1. オブジェクト所有者が報酬プールエクステンションを登録する。
2. 誰でもオブジェクトに添付された通貨固有の報酬プールを作成できる。

⁶Sui Foundation. Authenticated Events. Sui Documentation, 2025.

3. オブジェクトのファンドアキュムレーターに蓄積された収益が償還され、報酬プールに預け入れられる。
4. シェアトークン保持者がトークンをステークし、ステーク量に比例した報酬を請求する。

報酬プールは「オープン・ディストリビューション」モデルを使用する。いかなるシェア保持者も追加の認可なしにステークと請求ができる。これにより、エクステンション登録後の収益分配はパーミッションレスとなる。

例。 ある楽曲に 10,000,000 SHARE トークンが 3 人のソングライターに分配されている：Alice (5,000,000)、Bob (3,000,000)、Carol (2,000,000)。1,000 SUI の収益が楽曲の報酬プールに預け入れられると、Alice は 500 SUI、Bob は 300 SUI、Carol は 200 SUI を請求でき、ステーク量に比例する。

5.5 マルチカレンシー対応

各通貨タイプは独自の独立した報酬プールを受け取る。単一の楽曲または録音が、SUI、USDC、またはその他の Sui ネイティブトークンで同時に収益を蓄積・分配できる。これは報酬プール関数のジェネリック型パラメータを通じて実装されており、新しい通貨をサポートするためにプロトコルの変更は不要である。

6 デील・プロトコル

録音とリリースの接続は、ディールオブジェクト、すなわち録音所有者から特定のリリースに録音を含めるための明示的なオンチェーン認可によって仲介される。

録音所有者 —作成—> デील
 デील —消費—> トラック
 トラック —追加—> ディスク —追加—> リリース

図 2: デीलからリリースへの認可フロー

ディールは作成時に録音のメタデータ (楽曲 ID、収益分配率、音声インジェスタータイプ、楽曲の長さ、カバーアート) を捕捉し、ターゲットリリース ID とトラックレベルの収益分配率にバインドする。録音所有者は `RecordingAdminCap` を使用してディールを作成し、録音の含有に対する明示的な認可を提供する。

トラックが作成されると、ディールを消費し (ディールオブジェクトは破棄される)、認可とメタデータをトラック構造に転送する。トラックはディスクに整理され、リリースに組み立てられる。このフローは、リリースに録音を含めるためのライセンシングという現実世界のプロセスをモデル化する。マスター権保持者 (録音所有者) が特定の条件での使用を明示的に認可する。

交渉が不成立の場合、ディールは消費されずに破棄でき、オフチェーントラッキングのための `DealDestroyedEvent` を発行する。

7 分散型メディアストレージ

音声ファイル、歌詞、コード譜、スコア、カバーアートはオンチェーンに保存されない。代わりに、MusicOS は Sui 上に構築された分散型 Blob ストレージネットワークである **Walrus**⁷ 上のデータへの参照を保存する。各メディア参照は、Walrus ネットワークからデータを取得するために使用できる Walrus Blob ID を含む `WalrusData` 構造体である。

この関心の分離 (オンチェーンのメタデータと経済ロジック、オフチェーンのメディアストレージ) は、各データ型の異なる要件を反映している。

- **メタデータと経済** には強い一貫性、決定論的実行、永続的な不変性が必要である。これらの特性は Sui ブロックチェーンによって提供される。
- **メディアファイル** には低コストでの高帯域幅ストレージと取得が必要である。これらの特性は Walrus のイレイジャーコーディングストレージネットワークによって提供される。このネット

⁷G. Danezis et al. “Walrus: An Efficient Decentralized Storage Network.” arXiv:2505.05370, 2025.

ワークは、ストレージノードの最大3分の2が利用不能であっても耐障害性を維持しながら、4～5倍のレプリケーションファクターを達成する。

この組み合わせにより、公開された MusicOS オブジェクトは音楽作品の完全で自己完結的な表現となる。オンチェーンオブジェクトはすべてのメタデータと経済ロジックを含み、Walrus 参照は実際の音声および視覚コンテンツへのアクセスを提供する。

8 エージェント互換性とプログラマブル・ミュージック

8.1 DDEX メッセージングから宣言的実行へ

音楽産業における流通プロトコルの最も近い既存のアナログは DDEX (Digital Data Exchange)⁸ であり、レーベル、ディストリビューター、DSP がメタデータと配信指示を通信するために使用する XML ベースのメッセージング標準のセットである。DDEX は「メッセージ」（どの情報が交換されるか）を定義するが、「実行」（メッセージが受信されたときに何が起こるか、どのように検証されるか、どの下流プロセスが開始されるか）を定義しない。DDEX 交換の各参加者はメッセージの独自の解釈を実装しており、産業全体で不整合な振る舞いが生じている。

MusicOS はこのモデルを逆転させる。参加者が独自に解釈するメッセージを定義するのではなく、MusicOS は共有ランタイム上で決定論的に実行される「状態遷移」を定義する。楽曲の作成、クレジットの追加、録音の公開、収益の分配は、解釈されるべきメッセージではなく、実行されるべきトランザクションである。Sui ブロックチェーンは、すべての参加者が同じ状態と同じ実行セマンティクスを観察することを保証する。

メッセージングから宣言的実行へのこのシフトは、自動化に対して深遠な意味を持つ。

8.2 エージェントネイティブ・インターフェース

AI エージェントは、決定論的でプログラマ的にアクセス可能なインターフェースを必要とする。曖昧なメッセージングプロトコルをナビゲートしたり、不整合に実装されたワークフローを解釈したりすることはできない。MusicOS はまさにエージェントが必要とするものを提供する。

- **決定論的な状態遷移。** すべての関数は明示的な事前条件（ステートガード、認可チェック、制約検証）と事後条件（状態変更、イベント発行）を持つ。エージェントは関数を呼び出す前にその関数が何をするかを推論できる。
- **オンチェーン検証可能性。** すべての操作は検証可能なオンチェーン状態変更を生成する。エージェントは API レスポンスを信頼する必要がなく、状態を直接検証できる。
- **コンポーザブルなトランザクション。** Sui のプログラマブル・トランザクション・ブロックにより、エージェントはマルチステップのワークフロー（ディール作成、トラック作成、ディスク組み立て、リリース公開）を単一のアトミックトランザクションに構成できる。
- **イベント駆動型コーディネーション。** すべての重要な状態変更は型付きイベント（例：`CompositionPublishedEvent`、`ReleaseRevenueDistributedEvent`）を発行し、エージェントがプロトコルのアクティビティをサブスクライブし反応することを可能にする。

アーティストのカatalogを管理する AI エージェントは、自律的に次のことが可能である。収益蓄積の監視、分配のトリガー、ディールオブジェクトの作成と破棄によるディール条件の交渉、新しいリリースの公開。すべて MusicOS スマートコントラクトとの直接的な相互作用を通じて、またはより使いやすい体験のためにプロトコル上に構築された API レイヤーを通じて。

⁸DDEX. Digital Data Exchange Standards. <https://ddex.net>.

8.3 新しい流通フォーマットとしてのプログラマブル・ミュージック

これらの特性の収束（忠実なデータモデル、決定論的な経済、分散型ストレージ、エージェント互換性）は、現在の音楽産業には存在しないものを生み出す。「プログラマブル」な流通フォーマットである。

従来の産業では、音楽を流通させるとは、音声ファイルとメタデータのサイドカー（通常はDDEX XML）をDSPに配信することを意味する。音声はコモディティである。メタデータは緩く構造化され、不整合に解釈される。経済は不透明で仲介されている。

MusicOSでは、音楽を流通させるとは、その作品の「権威あるレコード」であるオンチェーンオブジェクトを公開することを意味する。誰が作成したか、誰が所有しているか、収益がどのように流れるか、どの音声・視覚コンテンツが関連付けられているか、どのエクステンションがその振る舞いを統治

しているか。オブジェクトは機械可読であり、独立して検証可能であり、経済的に自己実行される。音楽作品の創造的、経済的、法的コンテキスト全体を体現するプログラマブルなエンティティである。

これはAI生成コンテンツの時代において特に重要である。音声制作のコストがゼロに近づくとき、希少な資源は音声そのものではなく、それを取り巻く人間の創造的・経済的コンテキストである。プログラマブル・ミュージックはそのコンテキストをオンチェーンに捕捉し、それが記述する作品から不可分にする。

9 関連研究との比較

MusicOSは音楽産業テクノロジーの中でユニークなポジションを占めている。

特筆すべきは、AudiusのOpen Audio Protocol⁹である。これはDDEXメッセージング標準（ERN、MEAD、PIE）をprotobufシリアライズされたトラ

	MusicOS	Audius	Sound / Royal	DDEX
タイプ	プロトコル	プラットフォーム	プラットフォーム	メッセージング標準
データモデル	楽曲/録音/リリース	トラック	トラック (NFT)	XMLスキーマ
権利構造	BPS分配による完全な階層	簡素化	簡素化	包括的
拡張性	パーミッションレス	プラットフォーム制御	プラットフォーム制御	N/A
収益	オンチェーン、決定論的	プラットフォーム仲介	二次市場	参加者ごとの実装
ストレージ	Walrus（分散型）	IPFS / 独自ノード	Arweave / 中央集権	N/A
エージェント互換性	ネイティブ	API依存	API依存	多様
ブロックチェーン	Sui (Move)	Solana / Ethereum	Ethereum / L2s	N/A

表 1: MusicOS と既存の音楽産業テクノロジーの比較

⁹Open Audio Protocol. Global Music Database. <https://docs.openaudio.org>.

ンザクシオンにラップし、バリデーターネットワーク全体にブロードキャストすることで「グローバル音楽データベース」の構築を目指している。そのアプローチは、DDEX スキーマを設計の中心に据え、オンチェーンプロトコルを既存のメッセージング標準に準拠させることである。これは根本的な制約を伴う。プロトコルは、プログラマブルな経済実行ではなく、組織間メッセージングのために設計された標準の限界を継承する。DDEX スキーマは音楽メタデータを包括的に記述するが、実行可能な状態ではない。収益分配、所有権の強制、権利に基づくアクセス制御は、保存されたデータから導出されるオンチェーン操作ではなく、データベースが対処できない外部の関心事のままである。Audius の Solana ベースの Payment Router プログラム¹⁰がこのギャップを示している。受取人アドレスと金額のリストを呼び出し元が提供するパラメータとして受け取り、トークンを分配する、Global Music Database に保存された音楽メタデータとは無関係な汎用スプリッターである。呼び出し元はオフチェーンで分配率を計算し、入力として渡さなければならない。

Open Audio の Mediorium ストレージレイヤーはまた、音声ファイルの保存とリスナーへの配信という2つの異なる関心事を混同している。音声データの保存はドメイン固有ではなく（音声ファイルは他のデータと同様にバイト列である）、低遅延配信は CDN の関心事であり、専用インフラストラクチャが最適に処理する。\$AUDIO トークンで音声固有のストレージノードにインセンティブを与えることは、プロトコルのストレージ信頼性を単一トークンの経済に結びつける。MusicOS はストレージを独自の経済的セキュリティモデルを持つ汎用分散型ストレージネットワークである Walrus に委譲し、配信は専門インフラストラクチャが独立して対処できる別の関心事として扱う。

MusicOS はデータモデリングとレガシー相互運用性の両方に対して異なるアプローチを取る。時代遅れのメッセージング標準にプロトコルを準

拠させるのではなく、MusicOS は真に新しいオンチェーンデータモデル（プログラマブルな経済ロジック、強制可能なライフサイクル、決定論的な収益実行を持つ型付きオブジェクト）を構築し、オフチェーンミドルウェアを通じてレガシー業界インフラストラクチャとブリッジする。Nautilus 駆動の API がオンチェーン MusicOS オブジェクトと DDEX メッセージフォーマット間を変換し、オンチェーン証明を生成してプロトコルを従来の業界参加者（DSP、PRO、ディストリビューター）に馴染みのある標準でブリッジできる。これにより、最初から後方互換性のために表現力を犠牲にするのではなく、プロトコルの表現力を維持しながら後方互換性を提供する。

10 議論

10.1 信頼の前提

MusicOS は、Sui ブロックチェーンのセキュリティ保証を継承する。バリデーター（ステークベース）の最大3分の1がビザンチンであるという前提の下で、安全性と活性が保証される。この前提の下で、すべての MusicOS の状態遷移は決定論的で検証可能である。

エクステンション・システムは追加の信頼の前提を導入する。管理ケーパビリティの保持者は、登録する各エクステンションの信頼性の影響を評価し受け入れなければならない。生 UID アクセスを持つ悪意のあるエクステンションは、同じオブジェクト上の他のエクステンションのダイナミックフィールドに干渉する可能性がある。この信頼モデルは明示的であり、プラグインインストールを管理するシステム管理者の現実世界の責任を反映している。

インジェスターの主な役割は、Walrus のプロブ ID と音声ファイルの技術的プロパティ（チャンネル数、ビット深度、サンプルレート、サンプル数）の関係を証明することである。プロトコルはどのイン

¹⁰AudiusProject. Payment Router. <https://github.com/AudiusProject/apps/tree/main/solana-programs/payment-router>.

ジェスターがこの証明を行ったかを記録するが、インジェスターがどのように検証を行うかは規定しない。MusicOS はパーミッションレスであるため、誰でもインジェスターをデプロイでき、プロトコルはインジェスターの品質について判断しない。代わりに、インジェスターの身元（型名）が永続的に記録され、依拠する当事者が証明の信頼性を評価し、独自の信頼の判断を行えるようにする。

10.2 プロトコルの中立性

MusicOS は、管理レベルの関数、アップグレード権限、トークンベースのガバナンスを持たない不変パッケージとしてデプロイされる。プロトコルトークン、手数料抽出、デプロイ後にプロトコルの動作を変更できる特権的な役割は存在しない。これは意図的な設計上の選択である。MusicOS は、音楽産業の参加者間の信頼を歴史的に侵食してきた商業的利益相反から自由な、プログラマブルな音楽プリミティブの中立的プロバイダーとなることを意図している。

一方的に条件を変更し、増加する手数料を抽出し、自社ユーザーに対してデータの非対称性を利用するプラットフォームに慣れた産業にとって、中立性は採用の前提条件である。ガバナンスメカニズムとアップグレードパスを放棄することにより、MusicOS はいかなる当事者の将来の決定もプロトコルのルールを変更できないという保証を提供する。プロトコルは公共財である。誰でもその上に構築でき、誰もそれを制御せず、そのルールはすべての参加者に等しく適用される。これは特に機関レベルの採用において重要である。大手レーベル、出版社、著作権管理団体は、一つのスタートアップがエンゲージメントのルールに対するガバナンス権限を握るインフラストラクチャにリソースを投入する可能性は低い。不変でオーナーレスなプロトコルは、その摩擦を完全に排除する。

10.3 なぜ Sui なのか

MusicOS のコアプロパティは Sui のオブジェクトモデルに依存している。この依存関係を理解するこ

とで、同等のプロトコルが他の主要ブロックチェーン上で構築できない理由が明確になる。

EVM チェーン (Ethereum、Polygon、Arbitrum) および Solana では、デジタルアセットは NFT 標準 (ERC-721、ERC-1155、Metaplex) に従い、トークン ID をオンチェーンに保存する一方、アセットを定義するメタデータ (タイトル、クレジット、音声参照、所有権分配) はオフチェーン、通常は IPFS または中央集権的 API に存在する。これらのアセットを操作するスマートコントラクトは、トークン ID とその所有者にアクセスできるが、アセットのプロパティにはアクセスできない。収益分配コントラクトは録音のクレジットリストを読み取れず、ライセンスングコントラクトは楽曲の分配率を検査できず、マーケットプレイスコントラクトはオラクルやオフチェーンインデクサーに問い合わせない限りリリースのトラックリストを検証できない。このアーキテクチャはメタデータをプログラマブルな状態ではなく表示上の関心事として扱っており、アセット構造を推論して正しく実行しなければならない経済プロトコルにとっては致命的な制約である。

Sui のオブジェクトモデルはこの制約を排除する。すべての MusicOS オブジェクト (`Party`、`Composition`、`Recording`、`Release`) はフィールドが完全にオンチェーンにあり、スマートコントラクトから直接アクセス可能な型付き Move 構造体である。 `distribute_revenue` がリリースのトラックを反復処理する際、同一トランザクション内でオンチェーン状態から各トラックの BPS 分配率、楽曲 ID、録音 ID を読み取る。ディールが作成される際、プロトコルは録音のマスター音声インジェスター型、楽曲のシェア型、トラックのカバーアートを、すべてオンチェーンフィールドから、外部データ依存なしに読み取る。完全なオンチェーン状態は利便性ではなく前提条件である。オフチェーンメタデータに依存する収益分配は決定論的たり得ず、スマートコントラクトが読み取れない所有権構造はプログラマ的に強制できない。

同様に重要なのが、Sui のオブジェクトライフサイクルのサポートである。MusicOS オブジェクトは静的なレコードではなく、定義された状態を経て

進行する。**Composition** はドラフトからパブリッシュドに移行する。**Track** は **Unassigned** から **Assigned** に遷移する。**Deal** はホットポテトであり、作成されたのと同じトランザクション内で消費されなければならない。これらのライフサイクル遷移は、バイパス可能なアプリケーションレベルのチェックではなく、Move 型システムと Sui ランタイムによって強制される。EVM や Solana では、ライフサイクル強制はコンパイラ保証のない手動の状態マシンパターンを必要とし、線形型の不在により、ホットポテトフロー（オブジェクトがトランザクション完了前に正確に一度使用されなければならない）は表現不可能である。

オンチェーンデータとオンチェーンライフサイクルが組み合わさることで、経済プロトコルが実用的になる。MusicOS は両方を必要とする。収益分配を計算するためのデータと、ディールが履行され、トラックが正確に一度割り当てられ、楽曲がパブリケーション後に変更できないことを保証するためのライフサイクルである。両方のプロパティがネイティブに成り立つプロダクションブロックチェーンは Sui のみである。

10.4 オンチェーンコスト分析

いかなるオンチェーンプロトコルにおいても、リッチで完全に入力されたオブジェクトの作成コストが禁止的でないかは懸念事項である。MusicOS は寛大な制限（録音あたり最大 150 クレジット（各最大 10 役割）、100 ステム、20 プライマリーアーティスト、50 フィーチャードアーティスト、20 ディスクにわたるリリースあたり最大 255 トラック）を提供し、すべて Sui の最大オブジェクトサイズ 250 KB

以内である。これらの最大限界を行使するキッチンシンクテストのガスプロファイリングにより、コストは無視できるレベルであることが確認されている。

現実世界のパラメータを持つ一般的な録音（数人のクレジットされた貢献者、数本のステム、1～2 人のプライマリーアーティスト）のコストは 0.5 セント未満である。最も極端なケース、実際のプロダクションでは到達しないような最大限に入力された録音オブジェクトでさえ、約 21 セントである。オンチェーンメタデータサイズは実用上の制約ではない。

参考として、従来の法的インフラストラクチャで同等の機能セットを近似するコストを考えてみよう。単一の録音に対して同等の所有権・収益構造を確立するには（複数の権利保持者間の分配契約の起草と締結、著作権登録、ロイヤリティ分配のためのエスクローまたは集合管理の設定、PRO・MLC・ディストリビューターへの必要書類の提出）、法務費用、管理コスト、組織会費で日常的に数千から数万ドルがかかる。所有権シェアの移転には新しい契約が必要であり、多くの場合双方に法律顧問が必要である。収益フローの監査にはフォレンジック会計士が必要である。MusicOS では、これらの操作のすべて（不変の所有権レコードの作成、収益分配率のエンコード、シェアトークンの移転、決定論的な収益分配）が 1 セントの数分の一で実行され、完全な監査可能性が追加コストなしで組み込まれている。

10.5 制限とトレードオフ

録音における楽曲分配率の不変性。録音は作成時に楽曲の `split_bps` を捕捉する。楽曲の所有者が

シナリオ	ガスユニット	SUI	USD
一般的な楽曲（3 クレジット、デモ）	~3M	~0.0017	~\$0.002
一般的な録音（8 クレジット、5 ステム）	~5M	~0.0028	~\$0.003
一般的なシングル（1 トラック、2 クレジット）	~3M	~0.0017	~\$0.002
最大録音（150 クレジット、50 ステム、70 アーティスト）	~385M	~0.21	~\$0.21
最大リリース（255 トラック、20 ディスク、50 クレジット）	~23M	~0.013	~\$0.01

表 2: メインネットガス価格（550 MIST/ユニット、1 SUI = \$1 USD）でのガスコスト推定

後に分配率を変更したい場合(楽曲の公開前)、既存の録音は古い分配率を保持する。これは意図的であり(交渉された条件の遡及的変更を防止する)、録音が作成される前に分配率が確定されている必要がある。

オンチェーン紛争解決なし。 MusicOS には、所有権紛争、テイクダウン要求、著作権侵害申し立てを解決するメカニズムは含まれていない。これらは本質的にオフチェーンのプロセスであり、法的管轄権と人間の判断を伴う。MusicOS はそのようなプロセスが参照できる検証可能なデータ基盤を提供するが、それらを置き換えるものではない。

エクステンションの干渉。 生 UID アクセスモデルは、将来互換性と引き換えにエクステンションの干渉の可能性を受け入れる。MusicOS 上に構築される管理レイヤーは、エクステンションのホワイトリストと監査を通じてこれを軽減できる。

11 結論

音楽産業のインフラストラクチャは、物理的な流通と中央集権的な仲介の時代に構築された。デジタルの豊かさ、AI 生成コンテンツ、プログラマブルな経済的関係の時代に奉仕するために、有意義な進化を遂げていない。その帰結は測定可能である。数十億ドルの未マッチング・ロイヤリティ、不透明な収益パイプライン、そして音楽にその価値を与える創造的、経済的、社会的コンテキストのいずれも捕捉しない流通フォーマット(音声ファイル)。

MusicOS は代替的な基盤を提供する。音楽権利の完全な構造をパーミッションレスで拡張可能なオンチェーン・プロトコルにエンコードすることにより、MusicOS は音楽の所有権、帰属、経済を検証可能、決定論的、かつプログラマブルにする。MusicOS は音楽産業の参加者を置き換えることを目的とするのではなく、彼らが支える創造的な仕事にふさわしいインフラストラクチャを提供することを目的とする。

本プロトコルは2026年第2四半期にデプロイ予定であり、Apache 2.0 ライセンスの下で利用可能である。

参考文献

- [1] IFPI. Global Music Report 2024. International Federation of the Phonographic Industry, 2024.
- [2] U.S. Copyright Office. Mechanical Licensing Collective: Annual Report. Library of Congress, 2023.
- [3] G. Danezis, G. Giuliani, E. Kokoris Kogias, M. Legner, J.-P. Smith, A. Sonnino, and K. Wüst. “Walrus: An Efficient Decentralized Storage Network.” arXiv:2505.05370, 2025.
- [4] Sui Foundation. The Move Programming Language. 2024.
- [5] DDEX. Digital Data Exchange Standards. <https://ddex.net>.
- [6] Nautilus. Trustless Off-Chain Computation for Sui. <https://www.sui.io/nautilus>.
- [7] Sui Foundation. Authenticated Events. Sui Documentation, 2025.
- [8] SuiNS. Sui Name Service. <https://suins.io>.

MusicOS は Apache 2.0 の下でライセンスされたオープンソースソフトウェアである。本プロトコルは Subsonic Labs によって開発されている。